

Distribuirani sistemi

Komunikacija

Poziv udaljene procedure (Remote Procedure Call – RPC)

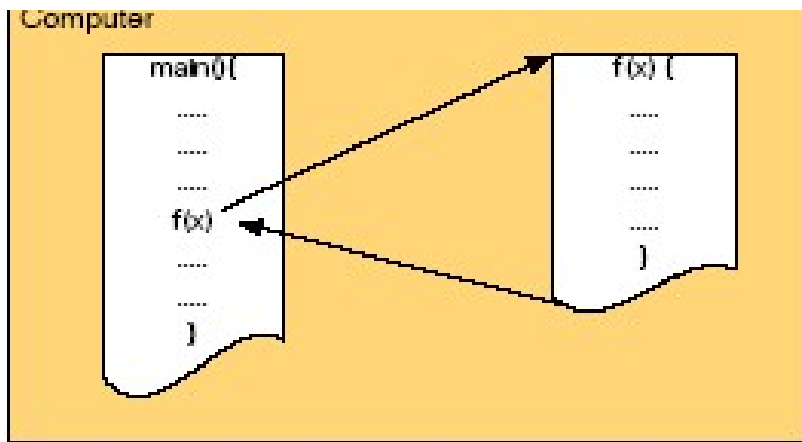
- * DS se baziraju na eksplicitnom slanju poruka između procesa.
 - Send/receive primitive ne skrivaju komunikaciju što je bitno da bi se postigla transparentnost pristupa
 - Problem je bio dugo poznat, ali se nije nalazilo rešenje
 - 1984. god Birell i Nelson su predložili potpuno novi način komunikacije:
 - Dozvoliti programima da pozivaju procedure koje su locirane na drugoj mašini:
 - Kada proces na mašini A pozove proceduru na mašini B, pozivni proces na mašini A se suspenduje, a izvršenje pozvane procedure se odvija na mašini B.
 - parametri procedure se prenose kao mrežne poruke između pozivajućeg i pozvanog programa
 - » za programera nikakvo slanje poruka nije vidljivo
 - Metod je nazvan Remote Procedure Call – RPC.
 - Klijent-server model: klijent pristupa udaljenom servisu pozivanjem odgovarajuće procedure koja implementira taj servis

RPC (nast.)

- * Ideja je jednostavna i elegantna, ali ima problema
 - Kako preneti parametre
 - Po vrednosti ili po referenci?
 - Klijent i server mašine mogu biti različite
 - Koriste različite formate za predstavljanje podataka;
 - Kako pronaći mašinu koja implementira udaljenu proceduru
 - Koja semantika važi u slučaju da nastupi greška:
 - obe mašine mogu otkazati, što može uzrokovati različite probleme;
- * Većinu problema RPC može da reši i danas je to široko korišćena tehnika na kojoj su bazirani mnogi DS.
 - SUN RPC, DCE RPC, gRPC, Remote Python Call (RPyC), Java RMI, MS DCOM, CORBA,,... su RPC sistemi

Kako RPC funkcioniše?

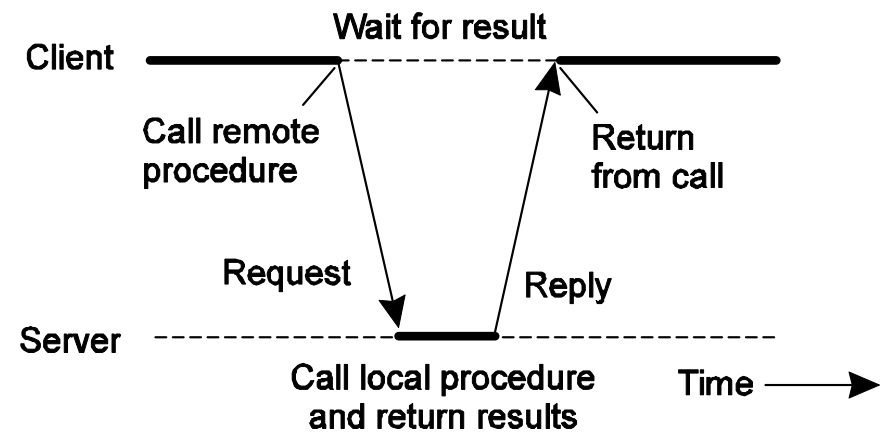
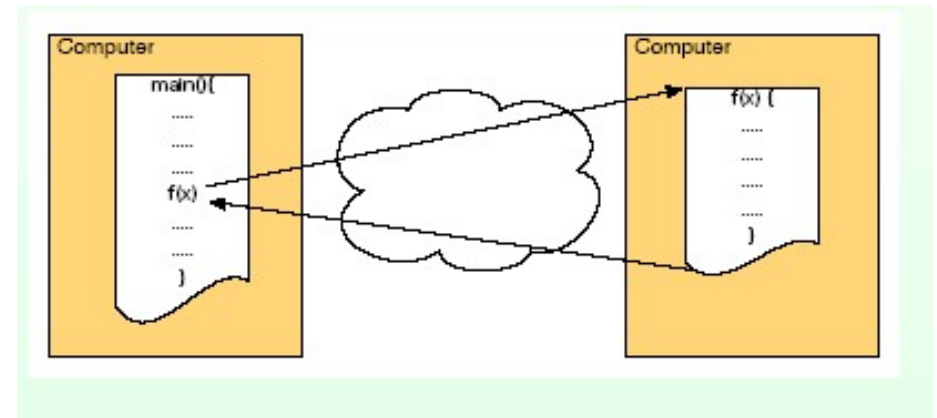
- Razmotrićemo prvo poziv konvencionalnih procedura, a zatim kako se poziv može razdeliti na klijentski i serverski deo koji se izvršavaju na različitim mašinama.



Poziv konvencionalne procedure:

- Nakon poziva procedure upravljanje se prenosi na pozvanu proceduru;
- na kraju izvršenja pozvana procedura vraća upravljanje glavnom programu.

I glavni program i pozvana procedura se izvršavaju na istoj mašini

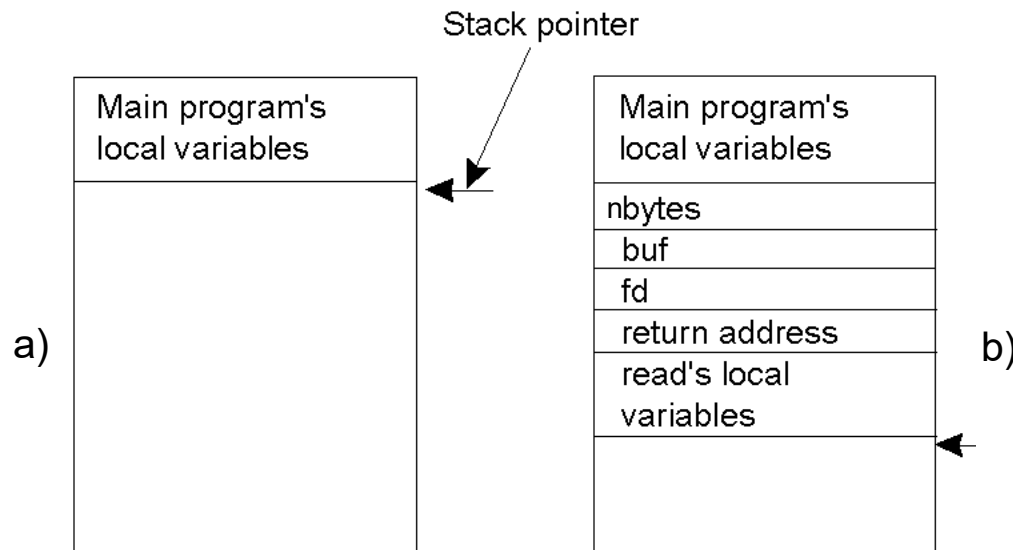


Poziv udaljene procedure

Poziv konvencionalnih procedura – prenos parametara

Primer

- `count=read(fd, buf, nbytes);`
- `fd`, je integer koji identifikuje fajl
- `buf`, polje u koje se učitavaju karakteri
- `nbytes`, integer koji kaže koliko bajtova treba pročitati
- Ako je poziv upućen iz glavnog programa, onda gl. program upisuje parametre procedure u stek (u redosledu prvo poslednji kod C-a)



Kada se `read` okonča, rezultat se upisuje u registar i upravljanje vraća glavnom programu.

Prenos parametara se može obaviti

- po vrednosti (`call-by-value`)
- po referenci (`call-by-reference`)
- `call-by-copy/restore`

Prenos parametara

* Prenos po vrednosti

- Vrednosti parametara se kopiraju u stek (npr. fd i nbytes)
 - Za pozvanu proceduru ovi parametri predstavljaju inicijalne vrednosti lokalnih promenljivih
 - Pozvana procedura može modifikovati ove vrednosti, ali to ne utiče na originalne vrednosti na strani pozivaoca.

* Prenos po referenci

- U stek se upisuje adresa parametra, a ne vrednost
 - U pozivu *read* drugi parametar se prenosi po referenci, jer se polja uvek prenose po referenci u C-u
- Ako pozvana procedura modifikuje preneti parametar, promeniće se i originalna vrednost na strani pozivaoca

* Call by-copy/restore

- Kod poziva procedure parametri procedure se kopiraju u stek kao kod poziva po vrednosti
- Nakon okončanja poziva, vrednosti se upisuju preko originalnih vrednosti parametara, kao kod poziva po referenci.
 - Ada kompajleri koriste copy/restore za inout parametre, a za ostale pozive po referenci.

* Koji način prenos parametara se koristi zavisi od programskoj jezika

- Nekada to zavisi i od tipa podataka koji se prenose
 - U C-u se skalarni tipovi prenose po vrednosti, a polja po referenci

Copy/restore

```
* Int y;  
* Call_procedure( )  
* {  
*     y = 10;  
*     Copy-restore(y); // L-value of Y is passed.  
*     Printf  y;      // prints 99  
* }  
* Copy-restore(x)  
* {  
*     x = 99;    // y still has the value of 10 (unaffected)  
*     y = 0;     // y is now 0  
* }
```

* Pre poziva f-je:	* Poziv f-je:	* Izvršenje f-je:	* Posle poziva f-je:
x	x=10	x=99	
y=10	y=10	y=0	y=99

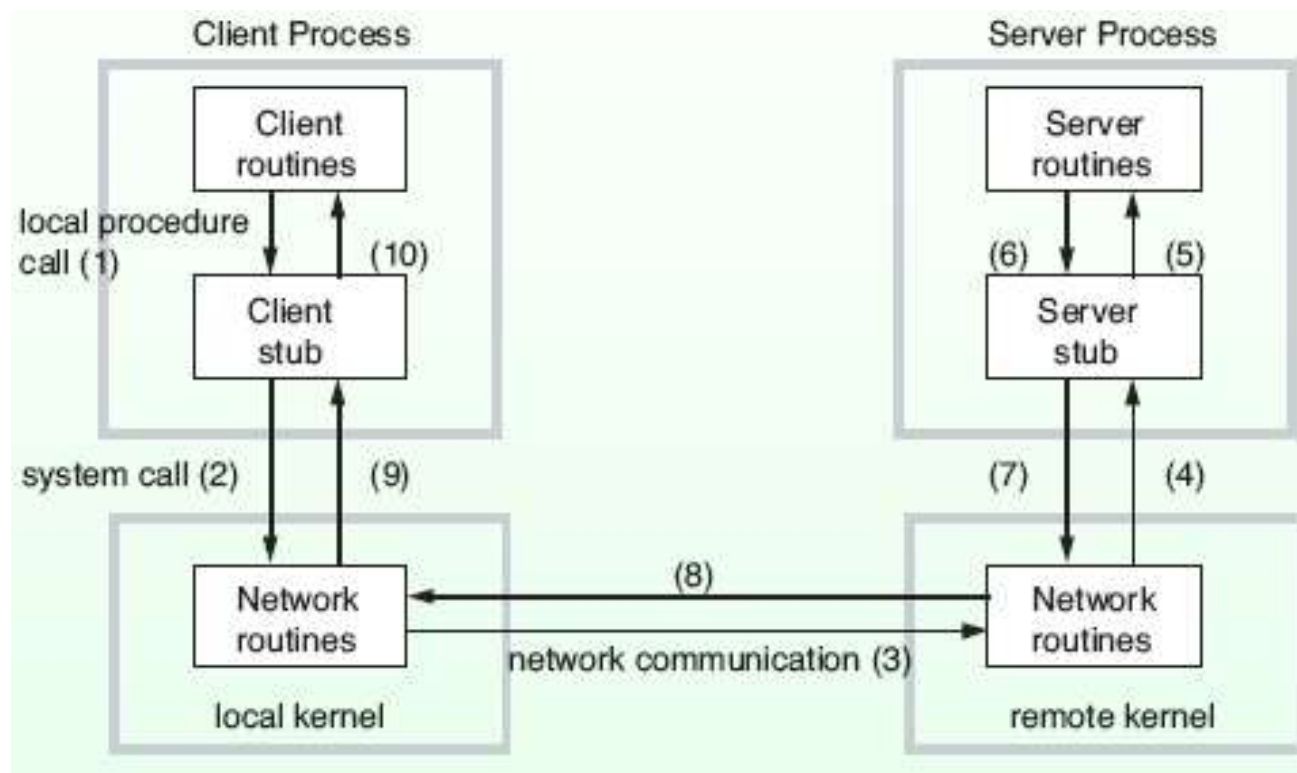
RPC (nast.)

- * Osnovna ideja RPC je da poziv udaljene procedure izgleda što sličniji pozivu lokalne procedure.
 - RPC obezbeđuje infrastrukturu neophodnu za transformisanje poziva procedure u poziv udaljene procedure na uniforman i transparentan način.
 - proces koji poziva udaljenu proceduru ne sme biti svestan da se pozvana procedura izvršava na drugoj mašini.
- * Primer
 - Pretpostavimo da program treba da pročita neke podatke iz fajla
 - Programer stavlja poziv *read* procedure u programski kod da bi pročitao podatke.
 - U jednoprosorskom sistemu *read* je bibliotečka procedura i njen kod se pomoću linkera ubacuje u objektni kod programa
 - To je kratka procedura koja je implementirana tako što se poziva ekvivalentni sistemski poziv *READ*.
 - *read* procedura je jedna vrsta interfeisa između korisničkog programa i lokalnog OS.
 - I ako *read* obavlja sistemski poziv, ona se poziva na uobičajeni način, smeštanjem parametara u stek (programer nije svestan da se u suštini obavlja sistemski poziv)

Klijent i server "stub" (nast.)

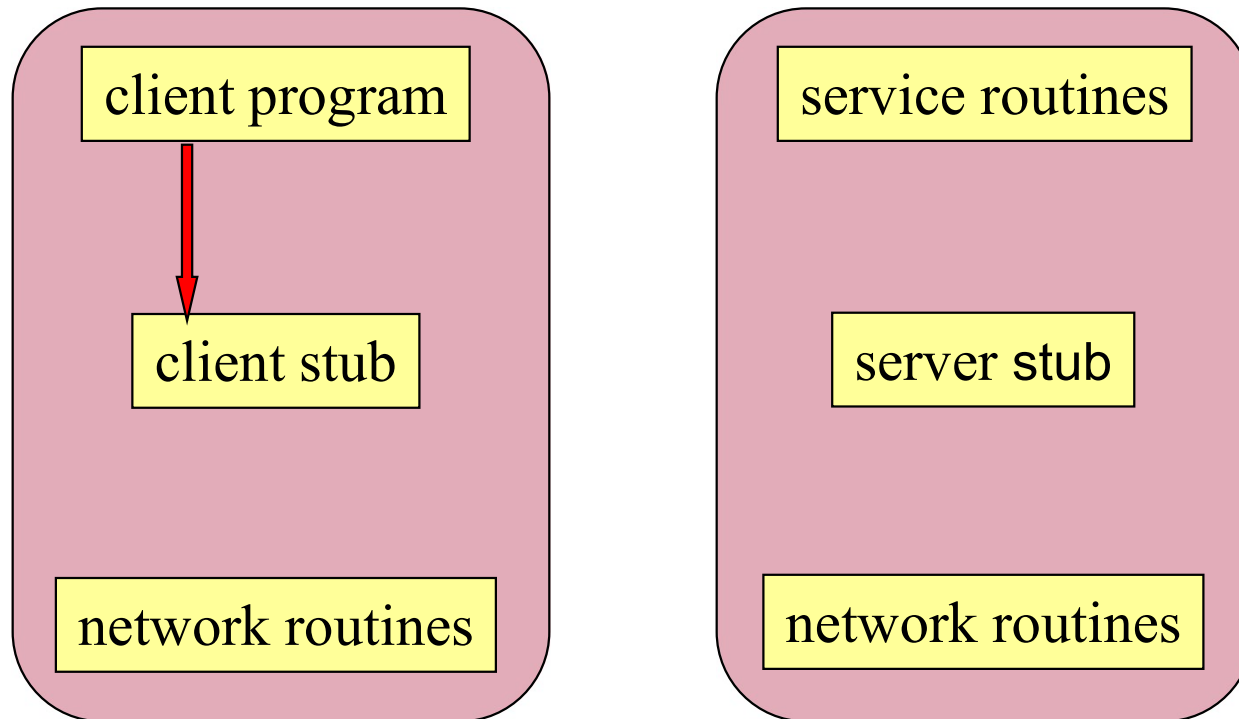
* RPC postiže transparentnost na sličan način

- Ako je *read* udaljena procedura (koja će se izvršiti na serveru) drugačija verzija *read*, koja se zove klijent stub se poziva, koja na osnovu primljenih parametara gradi poruku i poziva lokalni OS da prenese poruku do udaljenog servera.
 - Poziv stub funkcije izgleda kao poziv funkcije koju korisnik želi da pozove, ali ona stvarno sadrži kod za slanje i prijem poruka kroz mrežu.



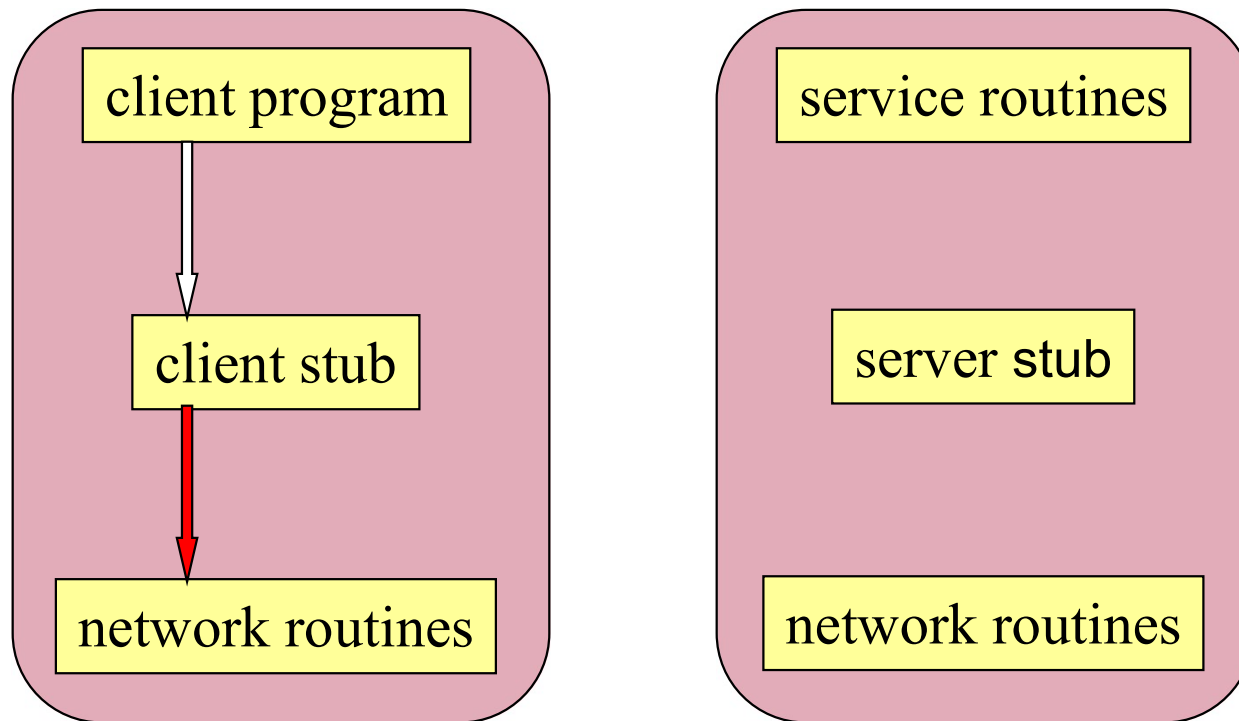
Koraci kod poziva RPC

1. Klijent poziva lokalnu proceduru (klijent stub) (parametri se smeštaju u stek)



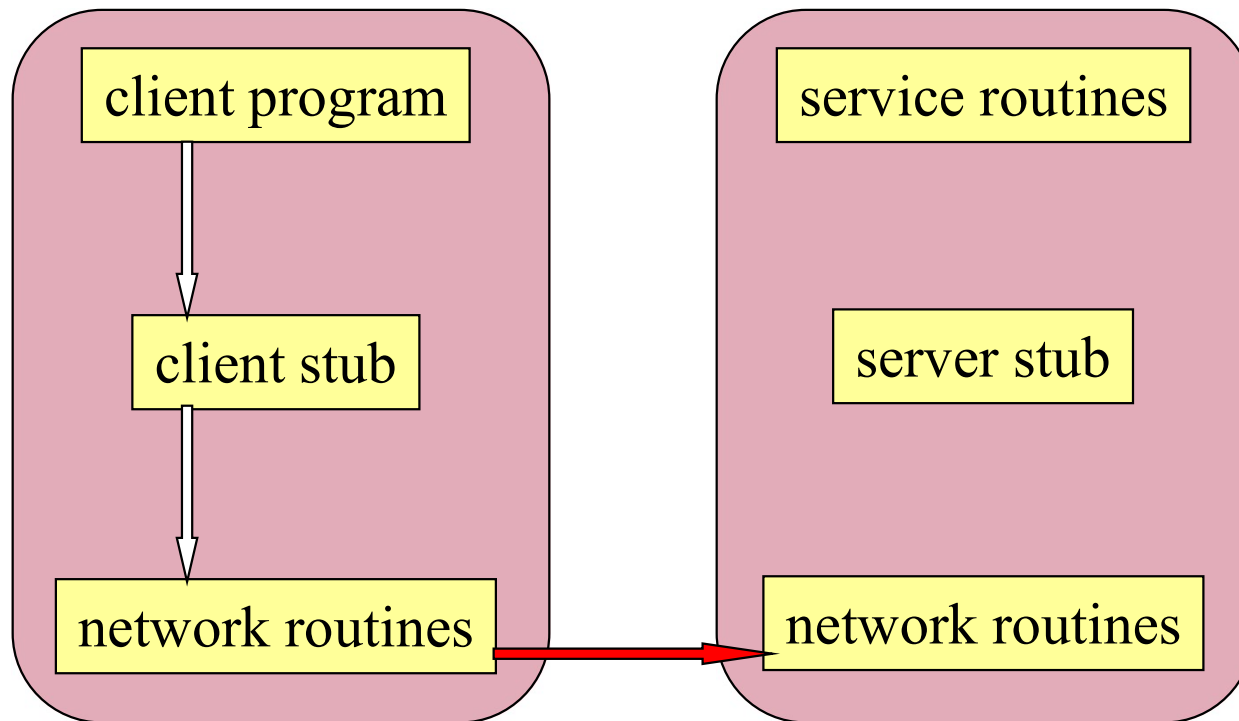
Koraci kod poziva RPC

2. Klijent stub pakuje argumente (marshaling) za udaljenu proceduru (ova aktivnost može uključiti i konverziju u standardni format) i gradi jednu ili više mrežnih poruka (messages) i poziva lokalni OS (poziva *send* primitivu, a nakon toga i *receive* primitivu i čeka se dok ne stigne odgovor).



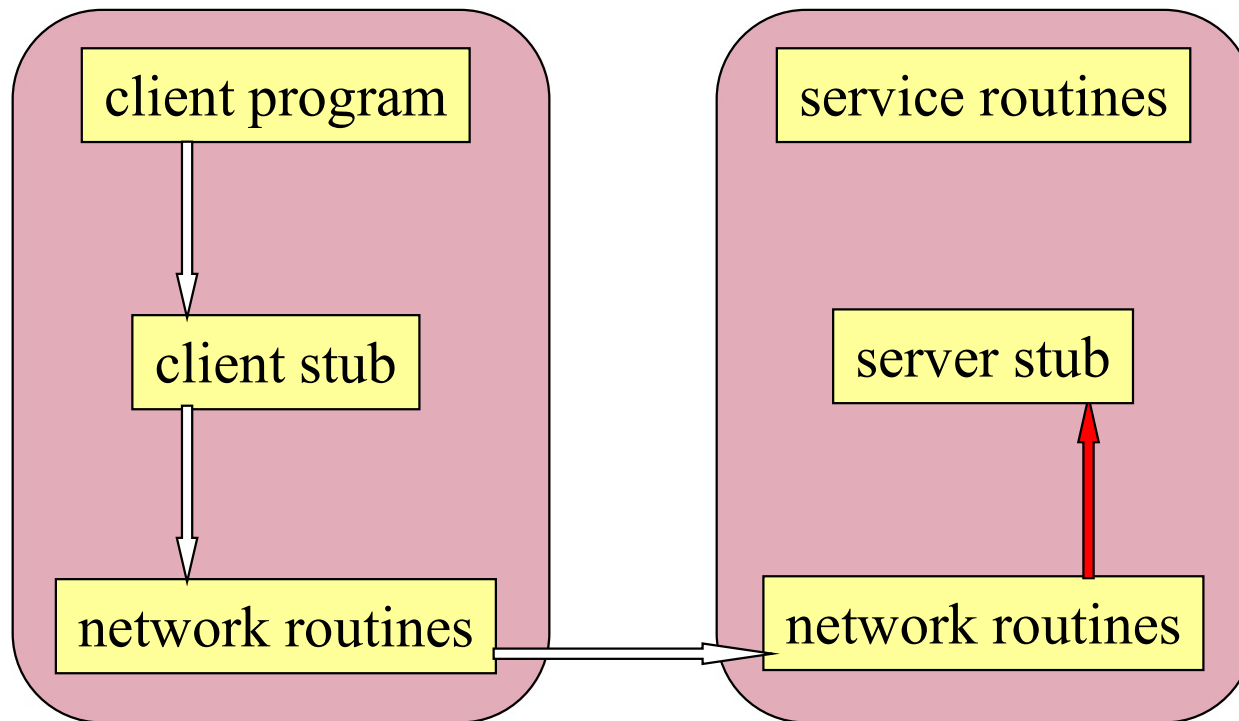
Koraci kod poziva RPC

3. Lokalni OS šalje poruku udaljenom OS (korišćenjem transportnog protokola -TCP ili UDP)



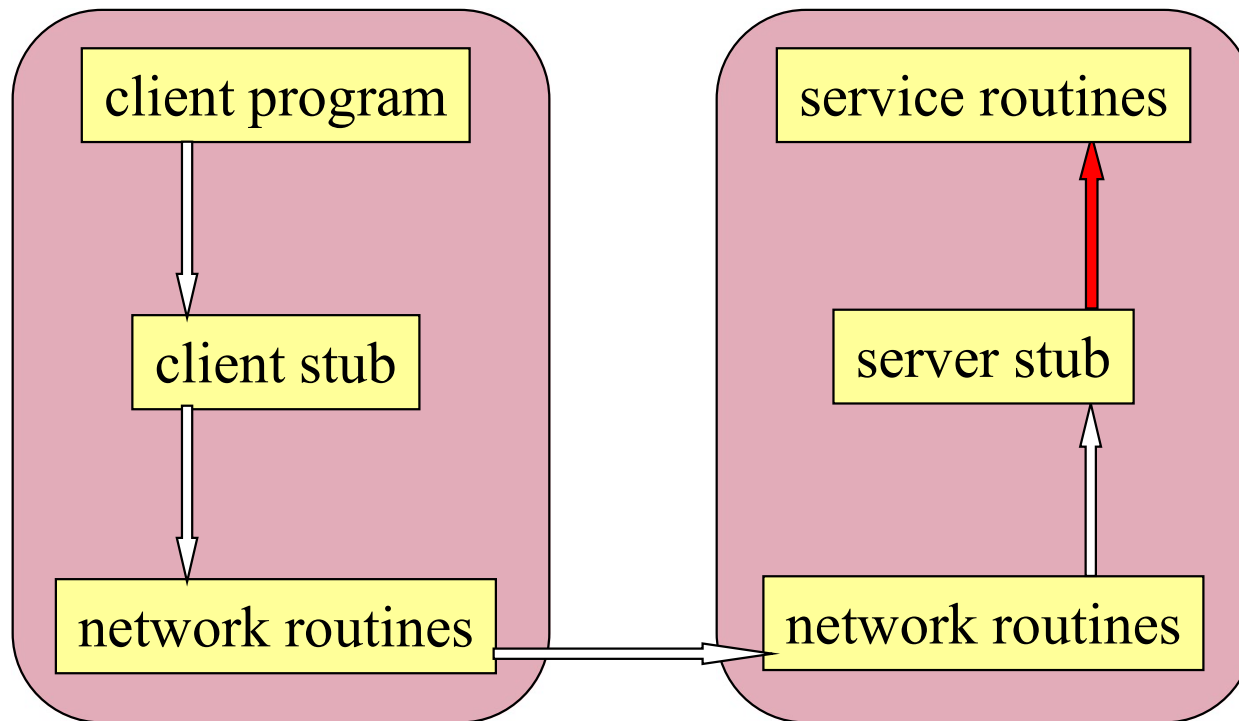
Koraci kod poziva RPC

4. Prijem poruke: mrežna poruka stiže do odredišta , OS prosleđuje poruku serverskom stub-u (na osnovu broja odredišnog porta). Serverska stub funkcija prima poruku od lokalnog OS (izvršava receive primitivu), raspakuje je (unmarshaling) i izvlači argumente za poziv lokalne procedure



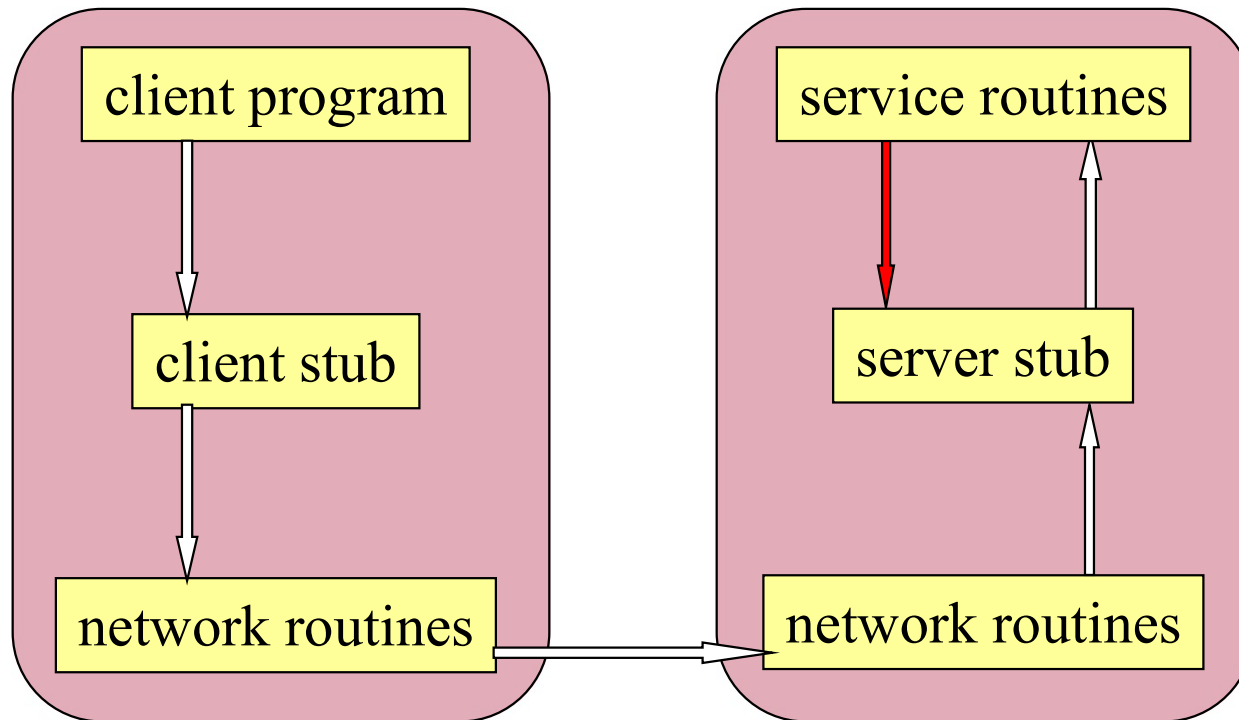
Koraci kod poziva RPC

5. Serverski stub poziva željenu serversku proceduru i predaje joj parametre koje je primio od klijenta (stavljajući ih u stek – kao kod poziva lokalne procedure).



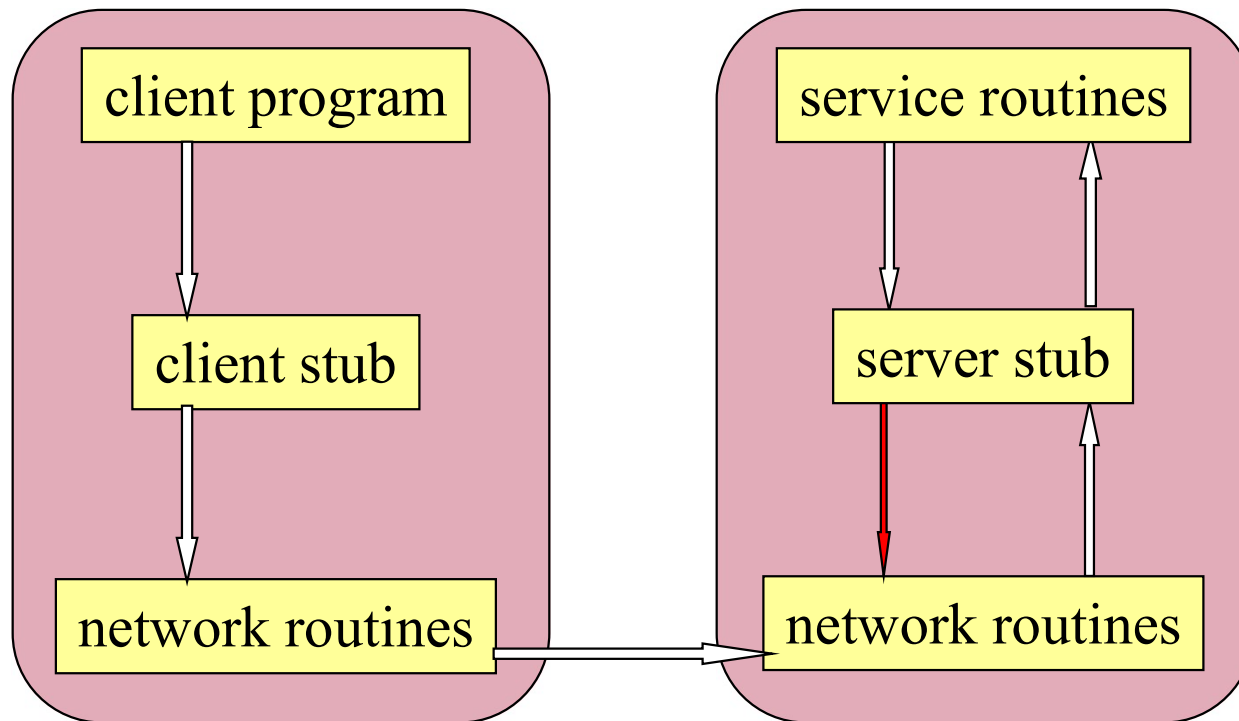
Koraci kod poziva RPC

6. Server izvršava pozvanu proceduru i vraća rezultat stub-u.



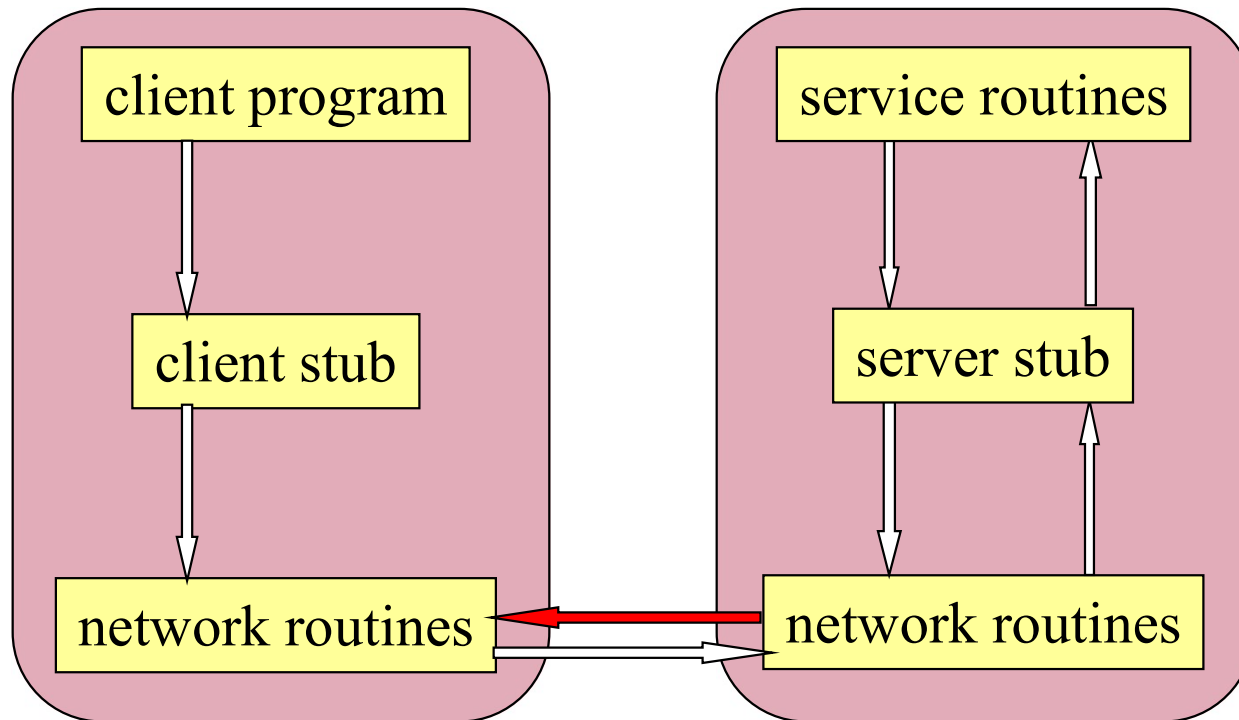
Koraci kod poziva RPC

7. Serverski stub pakuje rezultat u poruku i poziva svoj lokalni OS (poziva *send* primitivu, a nakon toga ponovo *receive* primitivu čekajući novi zahtev).



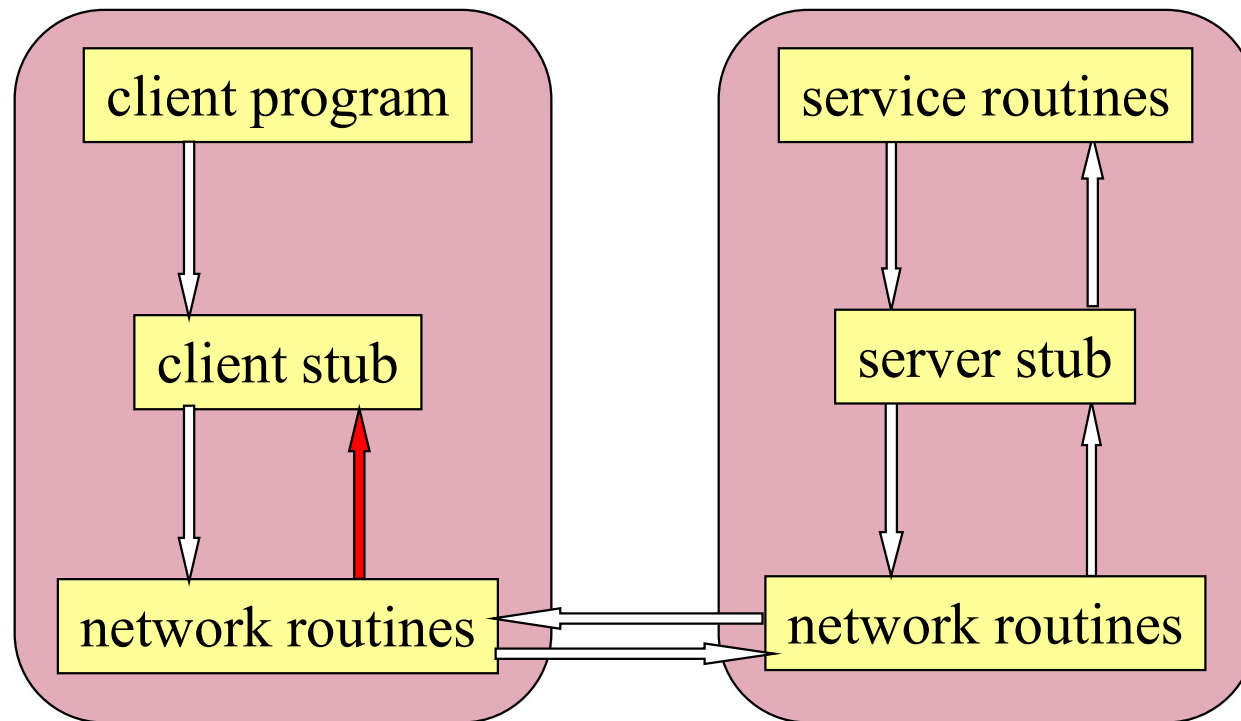
Koraci kod poziva RPC

8. Slanje poruke kroz mrežu: Serverski OS šalje poruku klijentskom OS



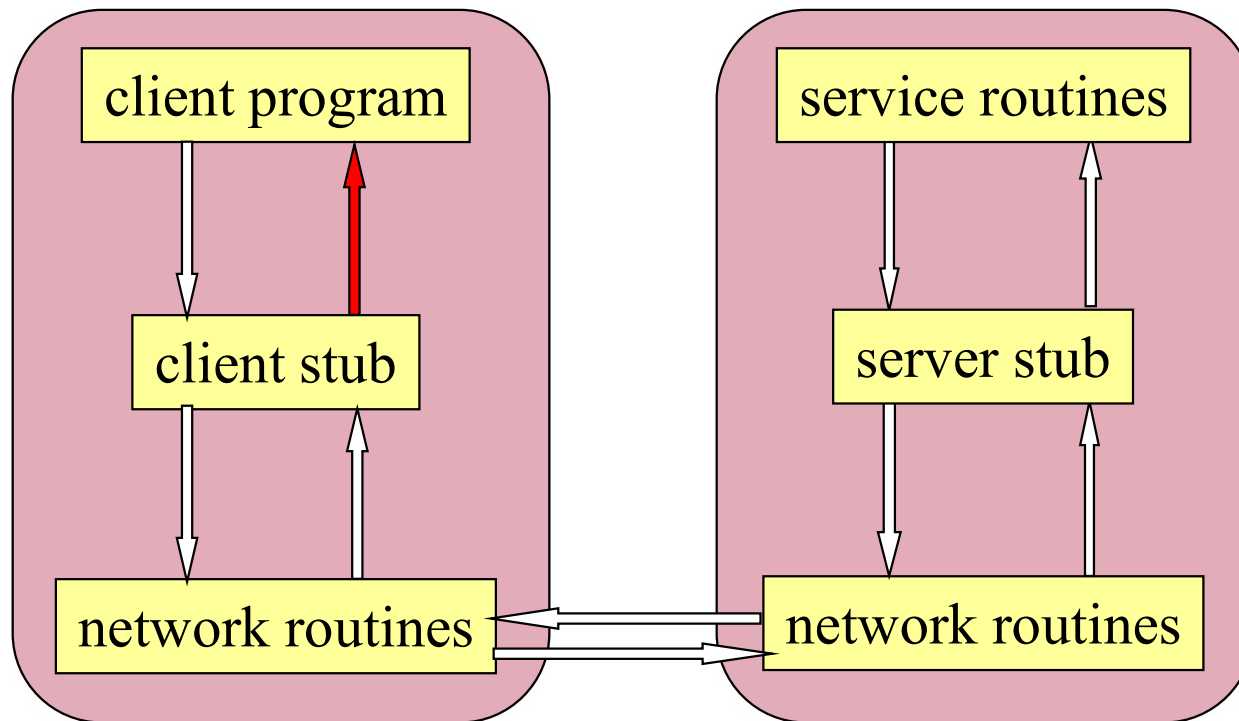
Koraci kod poziva RPC

9. Lokalni OS prosleđuje poruku klijnt stub-u



Koraci kod poziva RPC

10. Klijent stub izvlači rezultat iz poruke i vraća klijentskom procesu



10 koraka kod poziva RPC

1. Klijent poziva lokalnu proceduru (klijent stub)
 - Za klijentski proces ona izgleda kao aktuelna procedura koju treba pozvati.
2. Klijent stub pakuje argumente za udaljenu proceduru (ova aktivnost može uključiti i konverziju u standardni format) i gradi jednu ili više mrežnih poruka (messages) i poziva lokalni OS (poziva *send* primitivu, a nakon toga i *receive* primitivu i čeka se dok ne stigne odgovor).
 - Pakovanje argumenata u mrežnu poruku se zove "parameter marshaling" (uređivanje parametara)
3. Lokalni OS šalje poruku udaljenom OS (korišćenjem transportnog protokola)
4. Udaljeni OS prosleđuje poruku serverskom stub-u. Serverska stub funkcija prima poruku od lokalnog OS (izvršava *receive* primitivu), raspakuje je (unmarshaling) i izvlači argumente za poziv procedure.
5. Serverski stub poziva željenu serversku proceduru i predaje joj parametre koje je primio od klijenta (stavljajući ih u stek – kao kod poziva lokalne procedure).
6. Server izvršava pozvanu proceduru i vraća rezultat stub-u.
7. Serverski stub pakuje rezultat u poruku i poziva svoj lokalni OS (poziva *send* primitivu, a nakon toga ponovo *receive* primitivu čekajući novi zahtev).
8. Serverski OS šalje poruku klijentskom OS
9. Lokalni OS prosleđuje poruku klijnt stub-u
10. Klijent stub izvlači rezultat iz poruke i vraća klijentskom procesu
 - Klijent nastavlja dalje sa izvršenjem

- * Udaljenim servisima se pristupa uobičajenim pozivima procedura, a ne pozivanjem *send* i *receive*.
- * Svi detalji prosleđivanja poruka su skriveni u klijentskim i serverskim stub funkcijama, kao što su detalji stvarnih sistemskih poziva skriveni u bibliotečkim funkcijama.
 - Stub funkcije se kreiraju za svaku udaljenu proceduru

Prenos parametara kod RPC

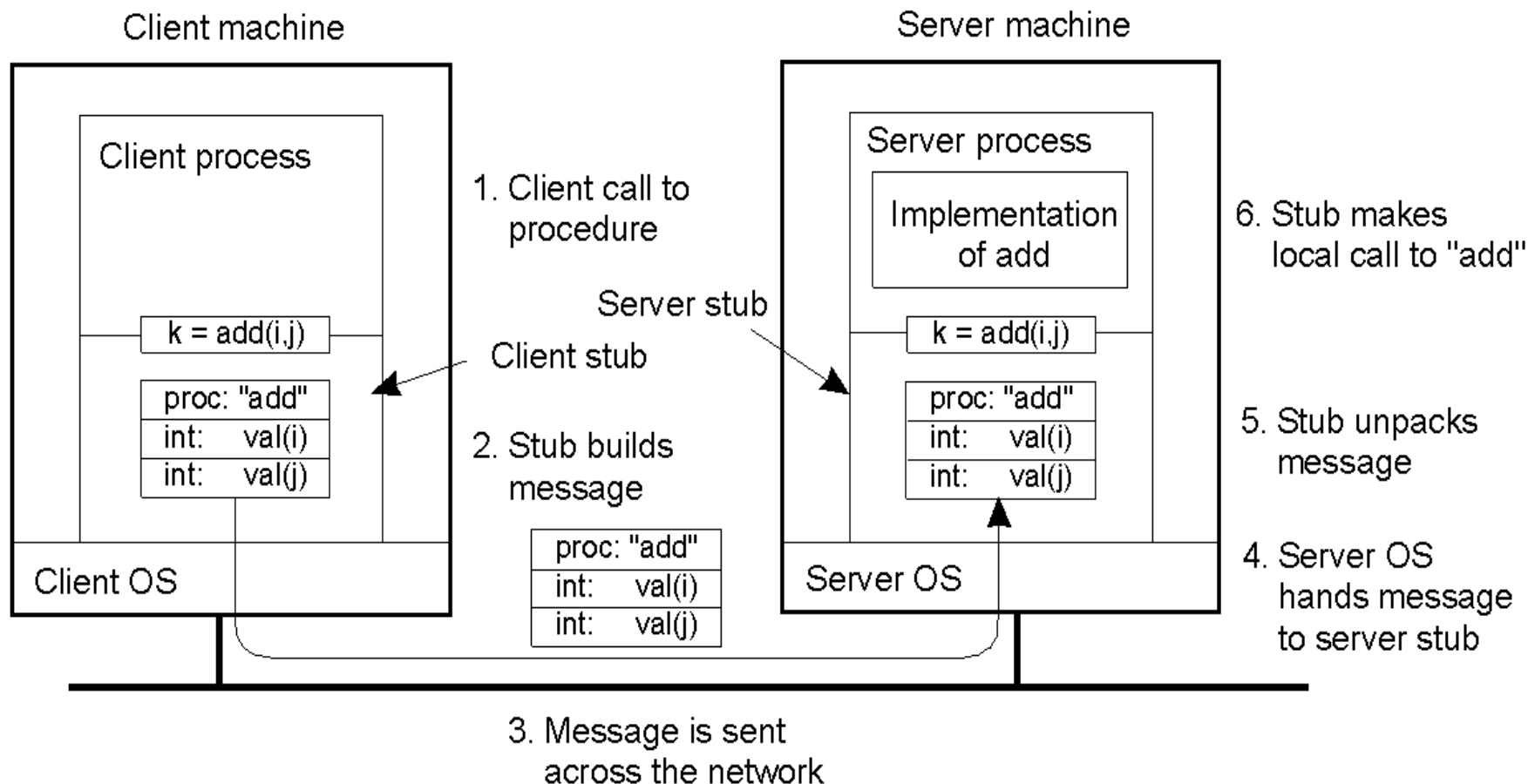
- Funkcija klijentskog stub-a je da preuzme parametre procedure od klijenta, spakuje ih u poruku (parameter marshaling) i pošalje serverskom stub-u.

* Kako se prenose parametri?

- Prenos po vrednosti je jednostavan
 - vrednosti se jednostavno kopiraju u mrežne poruke

* Primer

- Neka je `add(i,j)` udaljena procedura koja ima dva parametra tipa integer i kao rezultat vraća zbir dva integera.
- Poziv udaljene procedure `add` će prihvatiti klijent stub
- Klijentski stub uzima parametre i stavlja ih u poruku
- U poruku stavlja i ime ili broj procedure koja se poziva (`add`) jer server može podržati više poziva udaljenih procedura
- Kada poruka stigne do servera, serverski stub ispituje poruku da bi ustanovio koja procedura se poziva, i zatim upućuje odgovarajući poziv
 - Poziv koji stub upućuje serveru je sličan originalnom klijentovom pozivu, osim što su parametri promenljive koje se inicijalizuju iz dolazne poruke.
- Kada se procedura izvrši, serverski stub ponovo preuzima kontrolu:
 - Prihvata rezultat od izvršenja procedure, pakuje ga u poruku, šalje poruku klijentu
 - Klijentski stub raspakuje poruku i šalje rezultat klijentskoj proceduri.



- Sve dok su klijent i server mašina identične i dok su parametri skalarnog tipa ovo dobro funkcioniše
- Međutim, u DS je uobičajeno da postoji više tipova mašina:
 - svaka mašina može imati različit način predstavljanja brojeva (big-endian, little-endian, dvoični ili jedinični komplemet,...), karaktera (EBCDIC, ASCII)
 - Nije moguće preneti parametre procedure kao što je opisano
 - Problem se najčešće rešava tako koristi "standardno" kodiranje svih tipova podataka koji se mogu prenositi kao parametri.
 - Klijentski i serverski stub moraju obavljati konverzije iz lokalnog u standardni (i obrnuto) način predstavljanja podataka.

Prenos parametara

- Prenos po referenci je teško implementirati

- Nema nikakvog smisla proslediti adresu udaljenoj mašini.
- Ako se želi postići efekat prenosa po referenci, neophodno je iskopirati parametar (npr. polje) u poruku.
- Klijent stub kopira podatke na koje pokazuje pointer u poruku i poruku šalje serveru.
- Serverski stub koristi kopiju podataka, koristeći adresni prostor servera
- Serverski stub može pozvati serversku proceduru sa pointerom na podatak (npr. polje) koji je primljen i nalazi se u baferu ali će se vrednost pointera razlikovati od vrednosti na klijent strani.
- Modifikacije podataka koje serverska procedura uradi će direktno preko pointera uticati na sadržaj bafera u server stub-u.
- Kada server okonča izvršenje procedure, poruka će biti vraćena klijent stub-u, koji je zatim kopira na klijenta
- Ako je kopija izmenjena, onda će je klijentski stub vratiti klijentu, prebrisavši originalne podatke.
- U suštini je poziv po referenci zamenjen pozivom copy/restore.

Kako locirati udaljeni server i proceduru?

- * Da bi se realizovao poziv udaljene procedure neophodno je locirati udaljeni host i odgovarajući proces na hostu
 - Povezivanje (binding).
- * Dva rešenja su moguća
 - Statičko povezivanje: Klijent zna koji host treba da kontaktira (adresa hosta se nalazi u klijent stub-u), a kada klijent pozove odgovarajuću proceduru, klijent stub jednostavno prosledi poziv serveru.
 - Poseban program (portmapper) na udaljenom hostu pamti preslikavanja imena programa i broja verzije u broj porta
 - Prednosti ovakvog rešenja:
 - Jednostavnost i efikasnost, nema potrebe za dodatnom infrastrukturom osim klijent i server stub-a
 - Nedostaci
 - Klijent i server postaju čvrsto povezani (ako server otkaže, klijent neće moći da radi)
 - Ako server promeni lokaciju (IP adresu) klijent mora da se rekompilira sa novim stub-om koji ukazuje na pravu lokaciju

Kako locirati udaljeni server i proceduru? (nast.)

* Dinamičko povezivanje

- Drugo rešenje je da postoji centralizovana baza podataka (smeštena u name i directory serverima) koja može locirati host koji obezbeđuje željeni servis (rešenje koje su predložili Birell i Nelson)
- Ovi serveri imena i direktorijuma vraćaju adresu servera na osnovu "potpisa" procedure (imena i parametara) koja se poziva.
 - Kada klijent pozove udaljenu proceduru, klijentski stub kontaktira server imena da bi dobio adresu servera koji tu proceduru izvršava.
 - Server imena šalje adresu klijent stub-u koji zatim uspostavlja vezu sa željenim serverom.
- Ako server koji implementira željenu proceduru promeni adresu, dovoljno je promeniti ulaz u serveru imena
- Serverski stub registruje serversku proceduru u serveru imena i direktorijuma prilikom startovanja servera
- Veći overhead od statičkog, ali omogućava transparentnost i fleksibilnost

Problemi

- * kod poziva konvencionalne (lokalne) procedure ona će se izvršiti tačno jednom
 - ako nastupi greška ceo proces će biti uništen
- * kod poziva udaljene procedure postoji više mogućnosti za nastupanje grešaka
 - server može generisati grešku
 - mogu nastupiti problemi u mreži
 - server može otkazati
 - klijent može otkazati dok server izvršava pozvanu proceduru
- * Transparentnost se ovde završava!
 - aplikacija mora biti pripremljena za ove probleme (testirati greške)

semantika poziva udaljenih procedura

* kod poziva lokalnih procedura semantika je jednostavna

- procedura će se izvršiti tačno jednom kada je pozvana

* udaljena procedura može se izvršiti

- 0 puta, ako server otkaže pre izvršenja serverskog koda
- jednom, ako je sve ok
- jednom ili više puta, ako je komunikaciono kašnjenje veliko ili se izgubi odgovor od strane servera pa klijent izvrši retransmisiju

* Većina RPC sistema obezbedjuje

- "bar jednom" semantiku
 - ako su u pitanju idempotentne funkcije (mogu se izvršavati bez posledica više puta, npr. datum, vreme, čitanje statičkih podataka)
- "samo jednom" semantiku
 - funkcija nije idempotentna (poziv funkcije generiše efekte, npr. modifikovanje fajla)

Programiranje sa RPC

- Većina programskih jezika (C, C++, ...) ne podržavaju koncept udaljenih procedura i nemaju mogućnost da generišu klijent i server stub funkcije.
- Da bi se omogućilo korišćenje poziva udaljenih procedura iz ovih jezika, koristi se poseban kompajler koji generiše klijent i server stub funkcije na osnovu interfejsa procedura.
 - U klijent-server sistemu baziranom na RPC, lepak koji omogućava da se sve poveže u jedinstvenu celinu je definicija interfeisa napisana na IDL (Interface Definition Language).
- Definicija interfeisa je slična deklaraciji prototipa funkcije :
 - Ona sadrži set funkcija zajedno sa ulaznim parametrima i vrednostima koje se vraćaju.
- Kompajliranjem interfeisa pomoću RPC kompajlera generišu se klijentski i serverski stub-ovi
- Nakon što se obavi kompajliranje uz pomoć IDL kompajlera, mogu se kompajlirati klijentski i serverski programi i povezati (linkovati) sa odgovarajućim stub funkcijama.

Sun RPC

✱ Inicijalno projektovan za Sun OS, ali je danas raspoloživ za veliki broj platformi

- Sun RPC sistem obezbedjuje jezik za definisanje interfeisa (XDR – eXternal Data Representation) i interfeis kompajler (rpcgen) koji se koristi za generisanje klijent i server stub f-ja.
- rpcgen kompajler generiše tri fajla (npr. rpcgen primer.x)
 - header fajl (npr. primer.h)
 - Header fajl sadrži jedinstveni identifikator interfeisa, definiciju tipova, konstanti i prototipova funkcija.
 - On treba da bude uključen (korišćenjem `#include`) i u klijent i u server kod
 - klijent stub (**primer_clnt.c**)
 - Klijent stub sadrži procedure koje će klijent program pozivati
 - Ove procedure su zadužene za pakovanje parametara u poruke, slanje poruka, prijem poruka, izvlačenje rezultata poziva procedure i prosleđivanje klijentu
 - server stub (**primer_svc.c**) –
 - Sadrži procedure koje se pozivaju kada stigne poruka do servera i koje zatim pozivaju odgovarajuću serversku proceduru

Koraci kompilacije za RPC

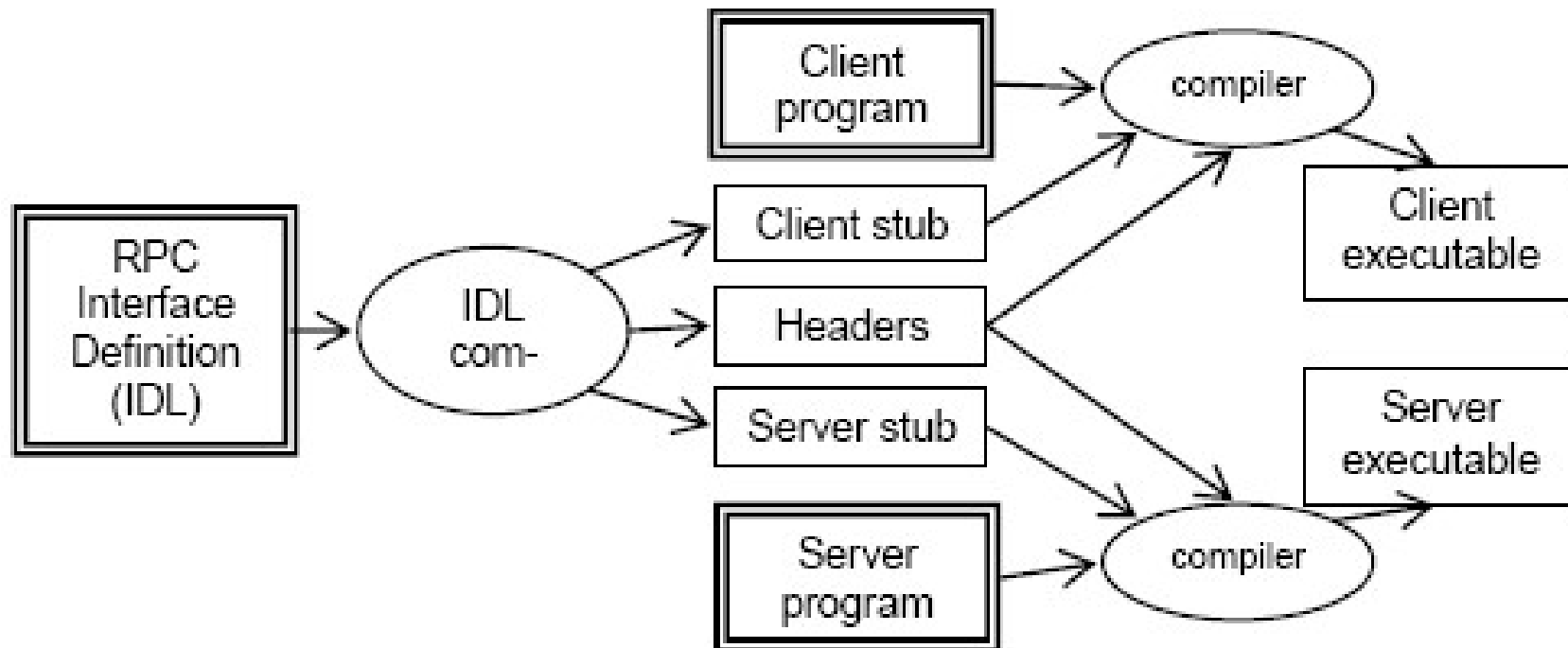


Figure 2. Compilation steps for Remote Procedure Calls

definicija interfeisa

- * Dva dela za opis mogu biti napisana u XDR i grupisana u **fajl** sa ekstenzijom **.x**
- * **XDR definicije**: definicije konstanti i tipova podataka parametara (ako nisu definisane u XDR-u)
- * **Definicija**: specifikacija procedura (usluga), odnosno identifikacija procedura i tipova podataka njihovih parametara
- * Prvi deo fajla file.x
 - XDR definicije **konstanti**
 - XDR definicije **tipova** ulaznih i izlaznih parametara za sve tipove za koje ne postoji ugrađena XDR podrška
- * Drugi deo fajla file.x
 - XDR definicije **procedura**
 - * Svaka procedura ima jedan ulazni i jedan izlazni parametar
 - * Identifikatori (imena) procedura se pišu velikim slovima
 - * Svakoj proceduri je pridružen **broj procedure**, jedinstven u okviru RPC programa

definicija interfeisa

- * svaka RPC procedura je definisana u okviru nekog programa i identifikovana je na jedinstveni način

- brojem programa – ASCII string predstavljen u Hex notaciji
 - identifikuje grupu udaljenih procedura, od kojih svaka ima svoj broj
- brojem verzije (programa)
 - svaki program ima i broj verzije, tako da ako se napravi izmena u udaljenom servisu (programu), npr. dodavanje nove procedure, programu se daje novi broj verzije, a ne novi broj programa
- brojem procedure
 - numeracija najčešće kreće od 1

* PRIMER

```
PROGRAM ime_programa {  
    def_verzije  
    def_verzije  
}= 0xYYYYYYYY
```

32-bitni identifikator

0x00000000-0x1fffffff: defined by sun

0x20000000-0x3fffffff defined by the user

0x40000000-0x5fffffff transient processes

0x60000000-0x7fffffff reserved

```
VERSION ime_verzije {  
    def_procedure1;  
    def_procedure2;  
    ...  
}=konstanta
```

definicija procedure

```
tip ime_procedure (tip_argumenta)  
=konstanta;
```

Primer: poziv lokalne procedure

```
#include <stdio.h>
int saberi();
int oduzmi();
main(int argc, char *argv)
{
    int a,b, rez1, rez2;
    if (argc!=3){
        poruka o gresci; exit(1);
    }
    a=argv[1]; b=argv[2];
    rez1=saberi(a,b);
    rez2=oduzmi(a,b);
    printf("zbir je=%d, razlika je = %d\n", rez1, rez2);
    exit(0);
}
int saberi( x, y){
    int x, y;
    return (x+y);
}
int oduzmi(x,y){
    int x, y;
    return (x-y);
}
```

Primer: Definicija interfeisa za pristup udaljenom servisu SABOD (Sun RPC)

* Sve udaljene procedure se deklarišu kao deo udaljenog programa

- PRIMER: servis KALK za sabiranje i oduzimanje dva cela broja, ima dve procedure:
 - SABERI (x,y)
 - ODUZMI (x,y)
- Kod Sun RPC poziv udaljene procedure može imati samo jedan argument

```
/* definicije tipova koji se prosledjuju
   udaljenim procedurama SABER i
   ODUZMI */
```

```
struct operandi {
```

```
int x;
```

```
int y;
```

```
};
```

```
/*definicija procedura i identifikatora
```

```
program KALK {
```

ime verzije

```
version KALK_VERSION {
```

```
int SABERI(operandi) = 1;
```

```
int ODUZMI(operandi) = 2;
```

```
}=1
```

broj verzije

```
}=
```

brojevi
procedura

broj programa

Sun RPC program

- ime programa, ime verzije, imena procedura u IDL fajlu se po pravilu pišu velikim slovima
- udaljene procedure koje su definisane pomoću IDL se u klijent programu pozivaju malim slovima navodjenjem imena procedure iza kojeg sledi donja crtica (_) i broj verzije
 - (imeprocedure_brojverzije)
- ove procedure u server programu imaju ime
 - imeprocedure_brojverzije_svc (malim slovima)
- Nakon kompajliranja IDL fajla komandom
 - rpcgen -C primer.x
- dobijaju se sledeći fajlovi
 - primer.h (header fajl)
 - primer_clnt.c (klijent stub)
 - primer_svc.c (server stub)

Pisanje SUN RPC programa

- Programer treba da napiše kod za:
 - Klijentski program: implementira `main()` funkciju i logiku potrebnu za pronalaženje udaljene procedure i povezivanje sa njom (fajl `primer_client.c`)
 - Server program: implementira udaljene procedure koje pruža RPC server (fajl `primer_server.c`)
 - Napomena: programer ne piše `main()` funkciju na serverskoj strani

Klijentska strana

- Klijent mora da zna:

- Ime udaljenog servera
- Informacije za pozivanje udaljene procedure: ime programa, broj verzije i ime udaljene procedure

- Za pozivanje udaljene procedure:

- Ime se malo menja: dodajemo _ nakon čega sledi broj verzije
- Dva ulazna parametra:
- Prvi ulazni parametar je predmet obrade procedure
 - Drugi parametar služi za uspostavljanje komunikacije se serverom
 - Izlazni parameter je pokazivač na rezultat
 - Omogućava mu da identifikuje neuspešan RPC (null povratna vrednost)

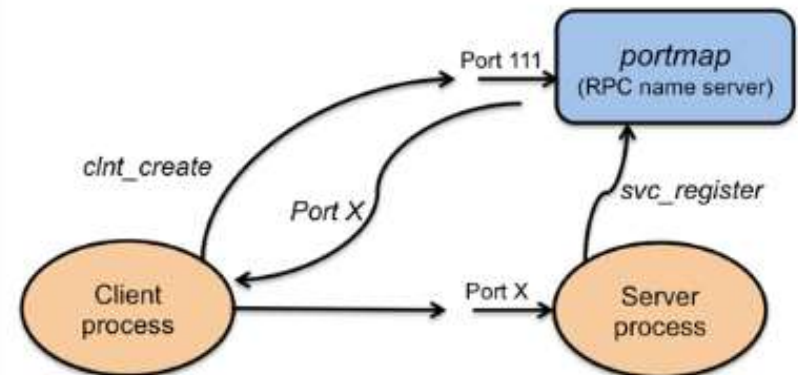
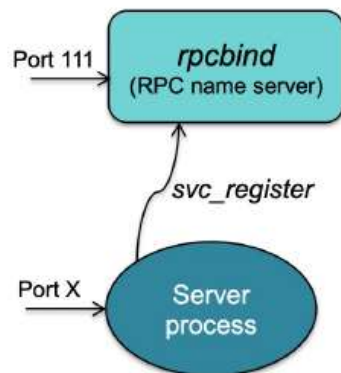
Serverska strana

- * Port mapper (rpcbind) osluškuje na portu 111
- * Prilikom izvršenja main() u serverskom programu, server stub kreira soket i povezuje ga sa dostupnim portom. Zatim registruje RPC procedure koje nudi u port mapperu (pozivom funkcije svc_register iz RPC biblioteke)
- * Port mapper održava dinamičku tabelu RPC servisa na toj host mašini
 - Svaka linija tabele sadrži trojku {prognum, versnum, protokol} i port

```
>$ rpcinfo -p
```

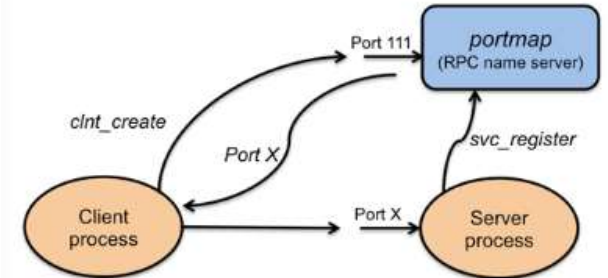
program	vers	proto	port	
100000	4	tcp	111	rpcbind
100000	4	udp	111	rpcbind
824377344	1	udp	59528	
824377344	1	tcp	49311	

- * Kada pokrenemo klijentski program, sa clnt_create se kontaktira port mapper na serverskoj strani kako bi se dobio odgovarajući port, pre nego što pozove udaljenu proceduru



Klijent strana

```
#include<stdio.h>
#include<rpc/rpc.h> /*standardna bibl za rpc */
#include "primer.h" /* fajl generisan pomocu rpcgen*/
int main(int argc, char *argv[])
{
    CLIENT *cln; /* RPC handle */
    char *server;
    int *zbir, *razlika; /* povratne vrednosti iz udaljenih procedura –vracaju se pointeri */
    operandi op; /*struktura definisana u IDL fajlu*/
    if (argc<2) {poruka o gresci; exit(1)}
    server=argv[1];
    cln =clnt_create(server, KALK, KALK_VERSION, "udp") /*kreira se klient handle i povezuje sa serverom*/
    if (cln==NULL) {poruka o gresci exit(2)} /*konekcija sa serverom nije uspostavljena */
    op.x=atoi(argv[2]); op.y=atoi(argv[3]);
    zbir=saberi_1(&op, cln); /*poziv udaljene procedure (u suštini poziva se stub funkcija)*/
    if (zbir==(int*) NULL){poruka o gresci; exit(3)}
    printf("suma je %d/n", *zbir)
    razlika=oduzmi_1(&op, cln) /*poziv udaljene procedure; argument procedure je pointer na argument čiji je tip
        definisan u IDL fajlu; procedura mora vratiti pointer na rezultat */
    if (razlika==(int*)NULL){poruka o gresci; exit(4)}
    printf("razlika je %d/n", *razlika)
    clnt_destroy(cln); /* zatvara se konekcija sa serverom */
    exit(0);
}
```



Server strana

* pomoću rpcgen može se generisati template za serverski kod korišćenjem prethodno definisanog interfeisa (u našem primeru primer.x)

- `rpcgen -C -Ss primer.x >server.c`

➤ server.c je ime fajla u kome je zapamćen serverski kod

- `rpcgen` će generisati sledeći kod

```
#include primer.h
int *saber_1_svc(operandi *argp, struct svc_req *rqstp)
{
    static int result;
    /*ubaciti serverski kod ovde*/
    return &result;
}
int *oduzmi_1_svc(operandi *argp, struct svc_req *rqstp)
{
    static int result;
    /*ubaciti serverski kod ovde*/
    return &result;
}
```

Server strana

```
#include<stdio.h>
#include<rpc/rpc.h> /*standardna bibl za rpc */
#include "primer.h" /* fajl generisan pomocu rpcgen*/
int *saber_1_svc(operandi *a, struct svc_req *rqstp)
{
static int zbir;
zbir=a->x+a->y;
return &zbir;
}
int *oduzmi_1_svc(operandi *a, struct svc_req *rqstp)
{
static int razlika;
razlika=a->x-a->y;
return &razlika;
}
```

Povezivanje klijenta i servera kod Sun RPC

- * klijent mora znati ime udaljenog servera
- * server registruje svoje usluge preko portmapper daemon procesa na serverskoj mašini (uvek na portu 111).
- * kompajliranje – linkovanje- izvršenje
 - generisanje stub funkcija
 - `rpcgen -C primer.x`
 - kompajliranje i linkovanje klijenta i klijent stub
 - `cc -o klijent klijent.c primer_clnt.c`
 - kompajliranje i linkovanje servera i server stub
 - `cc -o server server.c primer_svc.c`
 - Izvršenje
 - pozvati serverski program na izvršenje (recimo na hostu remus)
 - `$ ./server`
 - pozvati klijent program
 - `$ ./klijent -h remus 12 15`

Sun RPC definicije

- * RPC definicije tipova se kompajliraju i nalaze u odgovarajućem izlaznom header fajlu(example.h)
- * Postoji više vrsta definicija koje se mogu naći u fajlu .x.
Definicije nisu isto što i deklaracije jer se definicijom ne alocira nikakav memorijski prostor, već su to sam definicije jednog ili više elemenata. Promenljive će tek naknadno biti deklarirane.
- * Konstante:
 - `const DOZEN = 12; --> #define DOZEN 12`
- * Tipovi:
 - `typedef ST NT; --> typedef ST NT;`
- * Strukture:
 - `struct intpair { int a, b }; --> struct intpair { int a, b }; typedef struct intpair intpair;`

Sun RPC deklaracije

- * Deklaracije ne mogu stajati samostalno već moraju biti deo strukture ili u okviru typedef.
- * RPC podržava sledeće deklaracije:
- * Nizovi fiksne veličine:
 - `colortype palette[8];` --> `colortype palette[8];` (niz fiksne veličine od 8 celih brojeva)
- * Nizovi promenljive veličine:
 - `int heights<50>;` (maksimalne veličine 50 long vrednosti) ili
 - `long x_vals<>;` (max $2^{32}-1$ vrednosti)
 - PR. `typedef int heights <50>;` --> `struct { int heights_len; int *heights_val; } heights;`

gde je `heights_len` broj elemenata niza, `heights_val` pokazivač na prvi element niza

Pr. Interfejs za nalaženje prosečne vrednosti niza

Avg.x

```
const MAXAVGSIZE = 200;      /* maximum size of the array */

struct input_data {
    double input_data<MAXAVGSIZE>;
};

program AVERAGEPROG {
    version AVERAGEVERS {
        double AVERAGE(input_data) = 1; /* procedure number = 1 */
    } = 1;    /* version number */
} = 0x29999999;    /* program number */
```

Server strana

Avg_server.c

```
double *average_1_svc(input_data *input, struct svc_req *svc)
{
    static double sum_avg;
    double *dp = input->input_data.input_data_val;
    int i;

    sum_avg = 0;
    for (i=1; i<=input->input_data.input_data_len; i++) {
        sum_avg = sum_avg + *dp;
        dp++;
    }
    sum_avg = sum_avg/input->input_data.input_data_len;
    return(&sum_avg);
}
```

Klijent strana

Avg_client.c

```
void main(int argc, char* argv[])
{
    char *host;
    CLIENT *cInt;
    double *result_1, *dp, f;
    char *endptr;
    int i;
    input_data average_1_arg;

    host = argv[1];
    if(argc < 3) {
        printf("usage: %s server_host value ...\n", argv[0]);
        exit(1);
    }
```


Klijent strana (nastavak)

```
average_1_arg.input_data.input_data_val =  
(double*)malloc(MAXAVGSIZE*sizeof(double));  
    dp = average_1_arg.input_data.input_data_val;  
    average_1_arg.input_data.input_data_len = argc - 2;  
    for (i=1; i<=(argc-2); i++) {  
        f = strtod(argv[i+1], &endptr);  
        printf("value = %e\n", f);  
        *dp = f;  
        dp++;  
    }  
    clnt = clnt_create(host, AVERAGEPROG, AVERAGEVERS, "udp");  
    if (clnt == NULL) {  
        clnt_pcreateerror(host);  
        exit(1);  
    }  
    result_1 = average_1(&average_1_arg, clnt);
```

Klijent strana (nastavak)

```
if (result_1 == NULL)
    clnt_perror(clnt, "call failed:");
clnt_destroy(clnt);
printf("average = %e\n", *result_1);
}

}
```

SUN RPC prednosti

- * Aplikacija ne mora da vodi računa o dobijanju jedinstvene transportne adrese (broja porta)
 - Kod SUN RPC potreban je jedinstveni broj programa po serveru
 - Veća portabilnost
- * Aplikacija ne mora da vodi računa o veličini poruke, fragmentaciji, reasembliranju
- * Aplikacija treba da zna samo jednu transportnu adresu – adresu port mappera

Primer2: Distribuirano računarsko okruženje (DCE)

- * Distributed Computing Environment (DCE) – distribuirano računarsko okruženje, je middleware baziran na RPC.
 - Razvijen od Open Software Foundation (OSF), danas Open Group (OG)
 - to je skup servisa i alata koji se može instalirati na vrhu postojećeg OS, koji služi kao platforma za gradnju distribuiranih aplikacija
- * DCE je prvi middleware sistem koji je projektovan kao nivo apstrakcije između mrežnog OS i distribuirane aplikacije.
 - Inicijalno je projektovan za UNIX, ali sada postoji za sve veće OS
 - Ideja je bila da korisnik može na postojeći skup računara dodati DCE softver i nakon toga biti u stanju da izvršava distr. appl. ne narušavajući postojeće (nedistribuirane apl.)
 - Većina DCE sw se izvršava u korisničkom prostoru
 - Distribuirani fajl sistem mora biti dodat u kernel

DCE (nast.)

* Programski model na kome se baziran DCE je klijent-server.

- Korisnički proces se ponaša kao klijent da bi pristupio udaljenom servisu koji pruža serverski proces.
- Sva komunikacija između klijenta i servera se odvija pomoću RPC

* DCE servisi

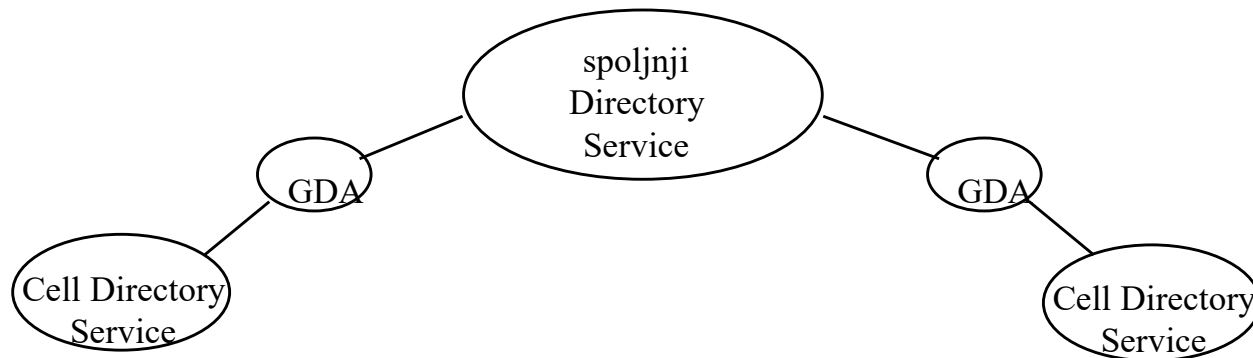
- RPC
- Direktorijumski servis
- Distribuirani fajl servis
- Bezbednosni servis
- Servis distribuiranog vremena

DCE RPC

- * fundamentalni komunikacioni mehanizam
 - svi ostali servisi koriste RPC za implementaciju svojih usluga
- * omogućava direktno pozivanje procedura na udaljenim sistemima, kao da su u pitanju lokalne procedure
- * pojednostavljuje pisanje distribuiranih aplikacija, eliminišući potrebu za eksplicitnim programiranjem mrežne komunikacije između klijenta i servera.
- * skriva razlike u predstavljanju podataka na različitim hardverskim platformama i omogućava distribuiranim aplikacijama da se izvršavaju na heterogenim sistemima

Direktorijumski servis

- * Servis koji na osnovu imena objekta vraća informaciju o imenovanom.
- * Npr:
 - ime osobe => broj tel. osobe
 - ime hosta => informacija o host (npr. IP adresa)
 - ime RPC servera => binding informacija za dati server
- * Efikasan direktorijumski servis je ključan u distribuiranom okruženju.
- * DCE Cell Directory Servis (CDS) je mehanizam koji omogućava korišćenje logičkih imena unutar DCE ćelije (grupa klijent i server mašina najčešće u okviru LAN)
- * aplikacije identifikuju resurse po imenu, bez potrebe da znaju gde je resurs lociran (za razliku od Sun RPC)
 - ovaj servis se koristi za lociranje servera na kome je implementirana udaljena procedura
- * DCE ćelije mogu međusobno komunicirati da bi locirali resurs koji je van lokalne DCE ćelije (slično kao što funkcioniše DNS)
- * integrisan u druge DCE servise i drugi servisi ga koriste
- * Zove se još i servis imena (*Name Service*)



Bezbednosni servis

- * Omogućava da resursi svih vrsta budu zaštićeni tako da pristup bude dozvoljen samo autorizovanim korisnicima
 - omogućava procesima na različitim mašinama da utvrde identitet onog drugog procesa (autentifikacija)
 - omogućava serveru da utvrdi da li je korisniku dozvoljeno da pristupi odredjenom resursu (autorizacija)

distribuiran fajl servis

- * DCE distribuirani fajl servis (DFS) pruža bezbedan metod za deljenje udaljenih fajlova
- * DFS korisniku izgleda kao lokalni fajl sistem, obezbedjuje pristup fajlovima sa bilo koje lokacije u mreži korišćenjem istog imena (uniformni pristup fajlovima)
- * DFS pruža mnoge usluge koje tradicionalni distribuirani fajl servisi ne pružaju, kao npr. keširanje, bezbednost, skalabilnost.

servis distribuiranog vremena

obezbedjuje mehanizme za sinhronizaciju časovnika na različitim mašinama u distribuiranom sistemu

- olakšava postizanje konzistentnosti u DS

DCE (nast.)

U klijent-server sistemu baziranom na RPC lepak koji sve drži na okupu je definicija interfejsa specificirana u IDLu.

- IDL omogućava deklaraciju procedura slično deklaraciji prototipa funkcija u C-u
- IDL fajl može sadržati definicije tipova, definicije konstanti, deklaracije procedura i druge informacije potrebne za korektno pakovanje i raspakovanje parametara.
- Ključni element u svakom IDL fajlu je globalno jedinstveni identifikator interfeisa.
 - Klijent šalje ovaj identifikator u prvoj RPC poruci i kontaktirani server proverava da li takav identifikator (interfeis) postoji

DCE (nast.)

* Prvi korak u pisanju klijent/server aplikacije je obično poziv `uuidgen` (universally unique identifier) programa od koga se traži da generiše prototip IDL fajla koji sadrži identifikator interfejsa i garantuje da se isti identifikator neće nigde pojaviti u DS generisanom sa `uuidgen`.

- Jedinstvenost se postiže kodiranjem i lokacije i trenutka kreiranja.

- Identifikator je 128-bitni broj predstavljen u IDL fajlu kao ASCII string u heksadecimalnoj notaciji

- NPR.

- `uuidgen imefajla.idl` (`uuidgen primer.idl`)
 - kreira se fajl `primer.idl` koji sadrži sledeće
 - ```
[
 uuid(3d6ead56-06e3-11ca-8dd1-826901beabcd),
 version(1.0)
]
interface INTERFACENAME
{

}
```

# DCE (nast.)

\* Sledeći korak je editovanje IDL fajla, tj. popunjavanje imena udaljenih procedura i njihovih parametara

\* Npr.

- sada treba zameniti INTERFACENAME imenom interfeisa
- Npr.
- ```
[  
  uuid(3d6ead56-06e3-11ca-8dd1-826901beabcd),  
  version(1.0)  
]  
interface math  
{  
  
  }
```

* sada treba deklarirati operacije (procedure) u okviru interfeisa

* Npr

```
[uuid(3d6ead56-06e3-11ca-8dd1-826901beabcd), version(1.0) ]  
  
interface math{  
  
    long get_sum([in] long first, [in] long second);  
  
}
```

semantika poziva procedura

* DCE podržava dve semantičke opcije kod poziva udaljene procedure

- "bar jednom" semantiku

- ako su u pitanju idempotentne funkcije (mogu se izvršavati bez posledica više puta)
- takve procedure se moraju označiti kao idempotent u IDL –u

- "samo jednom" semantiku

- funkcija nije idempotentna (poziv funkcije generiše efekte, npr. modifikovanje podataka)

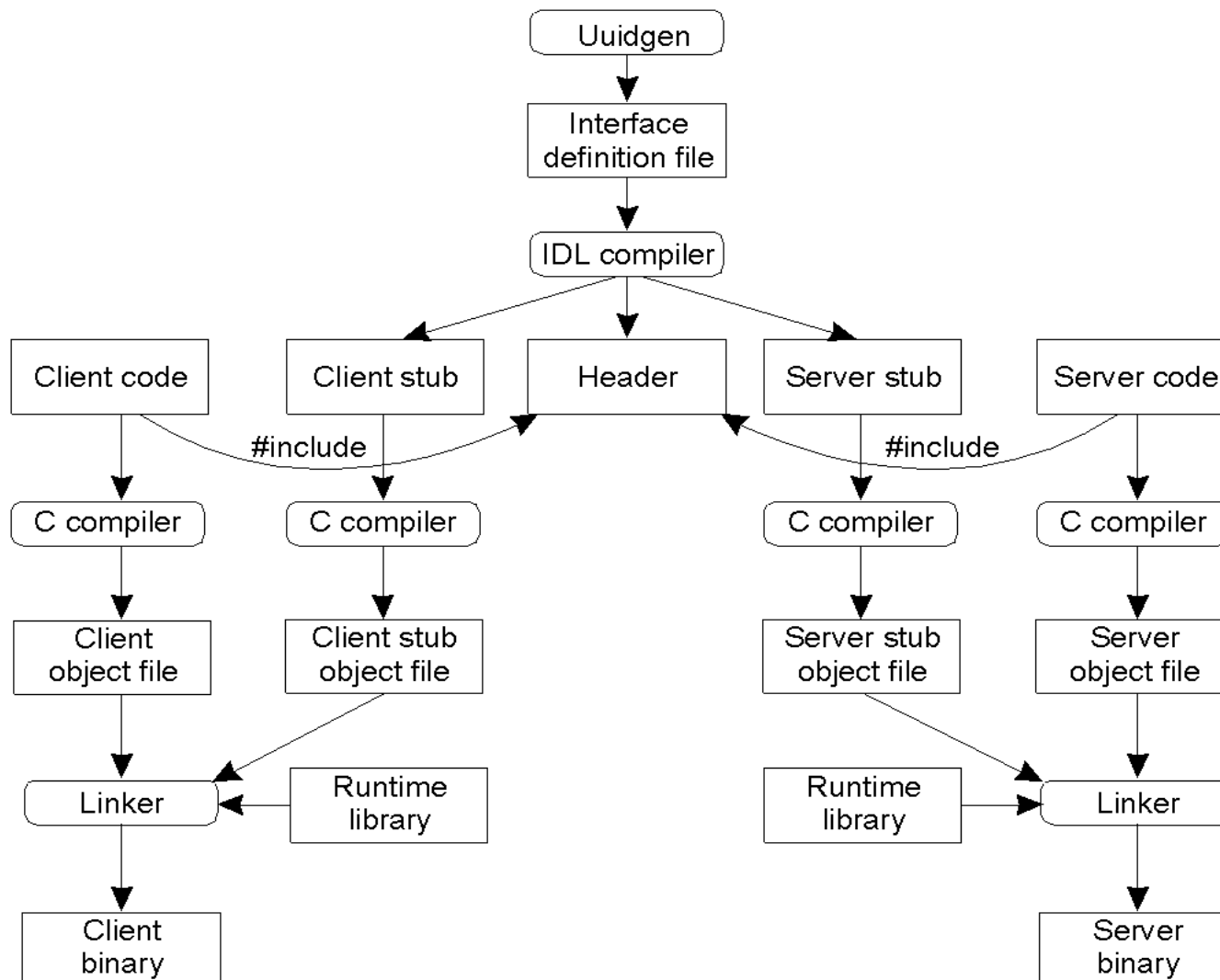
* Kada je IDL fajl potpun, poziva se IDL kompajler.

- `idl primer.idl`
- Izlaz iz IDL kompajlera sastoji se od 3 fajla
 - header fajla (npr. `primer.h`)
 - Header fajl sadrži jedinstveni identifikator, definiciju tipova, konstanti i prototipova funkcija.
 - On treba da bude uključen (korišćenjem `#include`) i u klijent i u server kod
 - klijent stub (**`primer_cstub.c`**)
 - Klijent stub sadrži procedure koje će klijent program pozivati
 - Ove procedure su zadužene za pakovanje parametara u poruke, slanje poruka, prijem poruka, izvlačenje rezultata poziva procedure i prosleđivanje klijentu
 - server stub (**`primer_sstub.c`**) –
 - Sadrži procedure koje se pozivaju kada stigne poruka do servera i koje zatim pozivaju odgovarajuću serversku proceduru

* Sledeći korak je pisanje klijent i server programa, a zatim kompiliranje ovih programa i klijent i server stub-a i povezivanje sa odgovarajućim stub-om da bi se dobio izvršni kod

• u opštem slučaju, DCE RPC aplikativni programer piše tri programa:

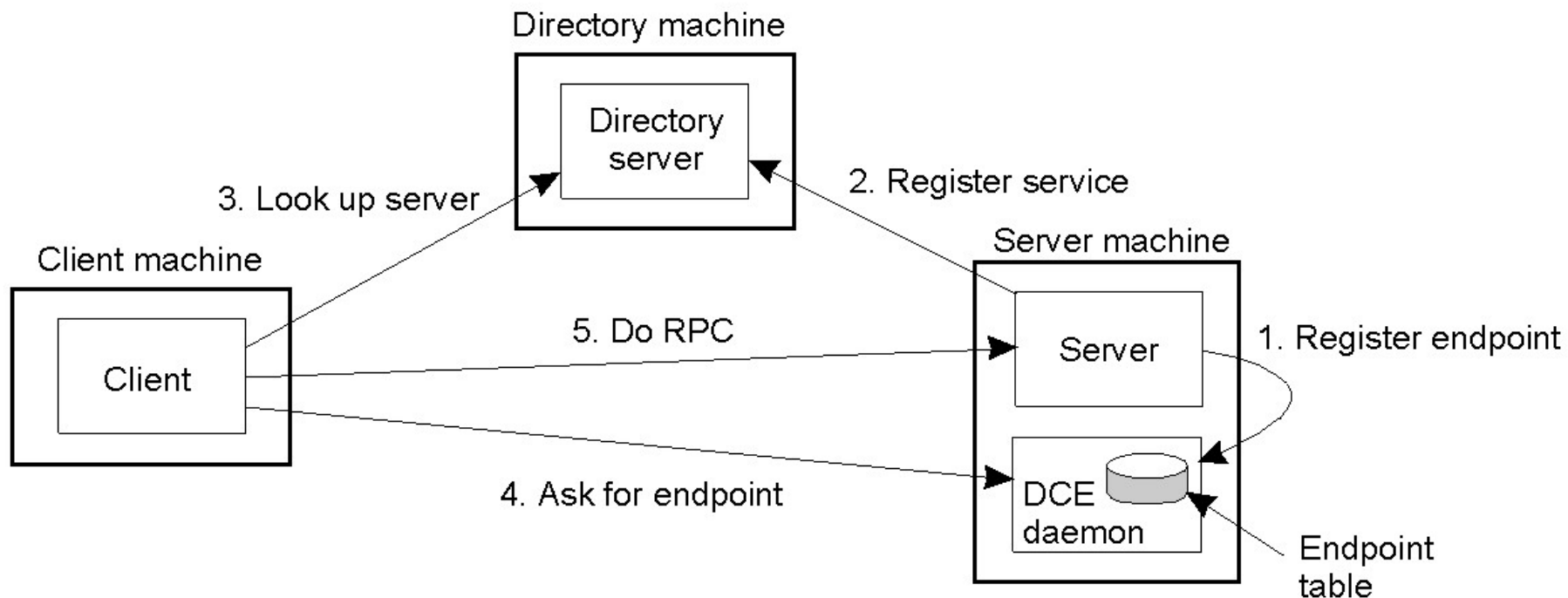
- klijent kod
- server inicializacijski kod kojim se server registruje u direktorijumskom serveru
- server operativni kod



Povezivanje klijenta i servera

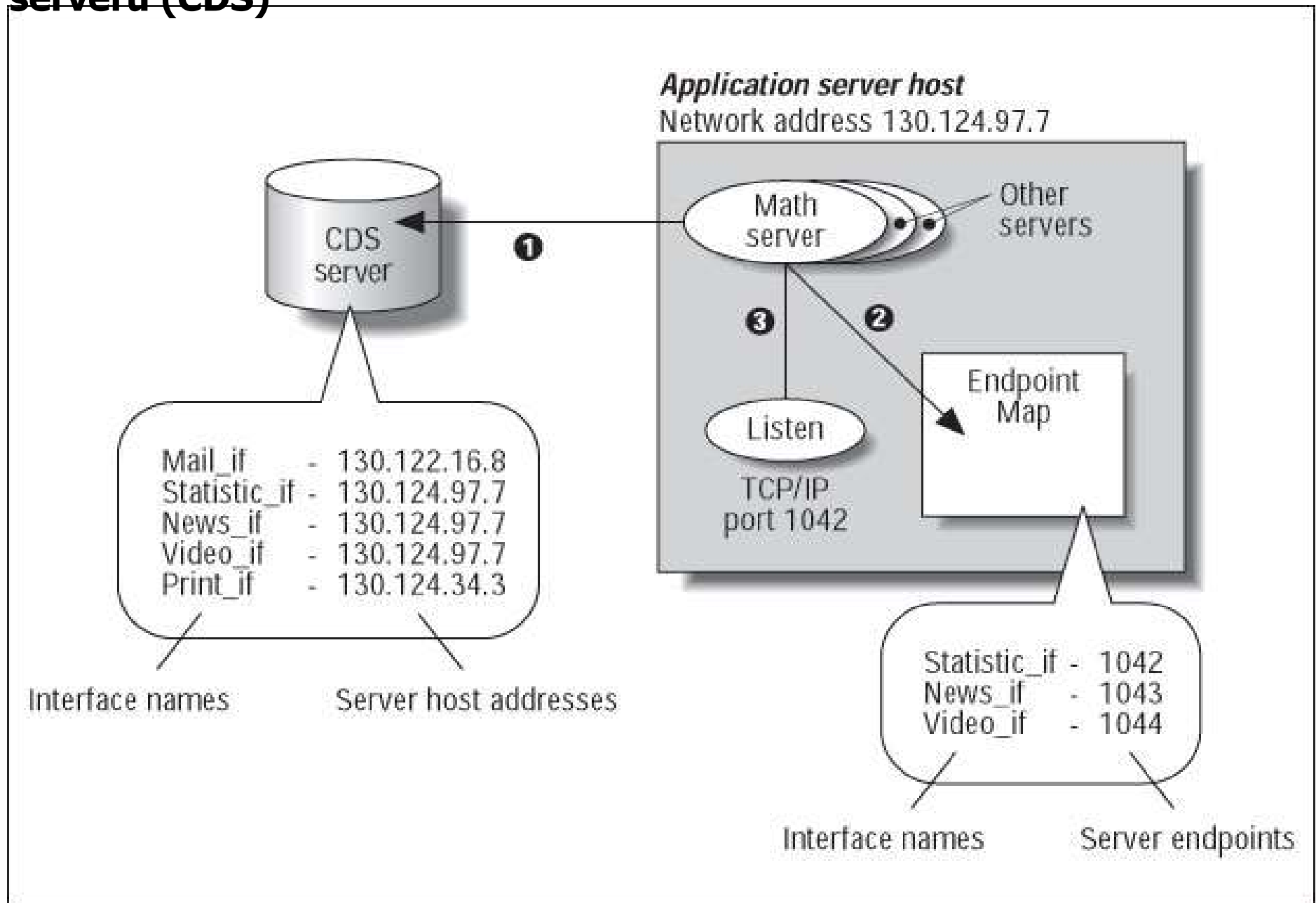
- * Da bi klijent mogao da poziva server, neophodno je da server bude registrovan i spreman da primi poziv.
 - Registracija servera omogućava klijentu da locira server i da se poveže sa njim
 - DCE klijent pronalazi server u dva koraka
 - Lociranje serverske mašine
 - Lociranje servera (tj. odgovarajućeg procesa) na serverskoj mašini.
 - Da bi klijent mogao da komunicira sa serverom (tj. procesom) on mora da zna broj porta (end point) na serverskoj mašini na koji može poslati poruke
 - Broj porta koristi serverski OS da bi prosledio poruku korektnom procesu (serveru)
 - U DCE-u na svakoj serverskoj mašini postoji tabela sa parovima (server, broj_porta) koju održava poseban proces, DCE daemon (*rpcd*) (koji ima poznati broj porta)
 - Server mora od OS da dobije broj porta koji se zatim registruje u DCE daemonu
 - Server se registruje i u **direktorijumskom serveru** tako što dostavlja ime serverske mašine i svoje interfeise

Povezivanje klijenta i servera



1. Registrovanje broja porta servera (procesa) u DCE deamonu
2. Registrovanje servera (procesa) u direktorijumskom serveru (adresa serverske mašine, ime servera i interfeisi koje implementira)
3. Klijent kontaktira direktorijumski server da dobije adresu server mašine (IP adresu) na kojoj se izvršava server (proces)
4. Klijent kontaktira DCE daemon da bi dobio broj porta servera(procesa) kome želi da pristupi
5. Poziv udaljene procedure

registrovanje servera i njegovih interfeisa u direktorijumskom serveru (CDS)



* pronalaženje Servera

